

UNIVERSIDAD TECNOLÓGICA DE PANAMA
CENTRO REGIONAL DE PANAMA OESTE
FACULTA DE INGENIERIA DE SISTEMAS COMPUTACIONALES
DESARROLLO DE SOFTWARE VI
PROYECTO No. 1

Resumen de arquitectura.

Este proyecto está conformado con un es la integración de una app móvil con dos opciones de backend (Local y Cloud).

La app selecciona automáticamente el backend mediante una bandera de configuración y utiliza repositorios para abstraer las funcionalidades principales: autenticación, panel de usuario, gestión de empleados, salarios e información personal.

El Stack de tecnologías utilizadas en este proyecto son:

PHP(backend Kotlin(app y backend cloud) MySQL Xampp PostgreSQL Cloud
)

Archivos de app

- config/: configuración de backend y proveedores.

BackendConfig y ProveedorDatos seleccionan PHP o Supabase.

- red/: Retrofit, endpoints y DTOs para consumo de API PHP.

- repositorios/: interfaces y adaptadores por backend.

- RepoAuthPhp / RepositorioAutenticacionSupabase.

- RepoPanelPhp / RepositorioPanelUsuarioSupabase.

- supabase/: cliente HTTP, configuración, APIs REST, hashing de contraseña.

- pantallas/: PantallaLogin, PantallaPanel y pantallas auxiliares.

- vistas/: ViewModels (ModeloVistaInicioSesion, ModeloVistaPanelUsuario).

- validacion/: validaciones de login y formulario de empleados.

- tema/: colores, tipografías y estilos.
- modelos/: DTOs y modelos de dominio.

Backend PHP

- config/:

conexion.php (driver mysql/pgsql), db_config.php (credenciales),
hasheo.php (hash), validaciones.php (reglas de datos).

- api/:

login.php, contraseña_cambiar.php,
empleados_registrar.php, empleado_editar.php, empleado_eliminar.php,
salarios_lista.php y endpoints relacionados.

Base de datos

- basededatos/personal.sql: esquema MySQL (XAMPP).
- basededatos/personal_postgres.sql: esquema para Postgres/Supabase.

Doble arquitectura de conexión a base de datos

La app soporta dos backends:

Opcion A: XAMPP + PHP (local)

UI -> ViewModel -> RepositorioAutenticacionPhp / RepoPanelPhp.

Retrofit (ClienteRed) apunta a http://10.0.2.2/ProyectoVI/backend_php/.

PHP valida datos (validaciones.php), consulta DB y devuelve JSON.

Acciones críticas usan transacciones (registro/edición/eliminación).

Opcion B: Supabase (REST)

UI -> ViewModel -> RepositorioAutenticacionSupabase /
RepositorioPanelUsuarioSupabase.

SupabaseClient agrega headers apikey y Authorization (SupabaseConfig).

SupabaseApi consume REST /rest/v1/usuario, /empleados, /salario, /inactivos.

Eliminar empleado crea registro en inactivos y luego elimina usuario y empleado.

Como funciona la conexion con Supabase

- SupabaseConfig define URL y API key.
- SupabaseClient arma el cliente HTTP con headers requeridos.
- SupabaseApi declara endpoints REST.
- RepositorioAutenticacionSupabase valida credenciales:
 - busca usuario por cedula,
 - verifica hash con SupabaseHashContrasena,
 - retorna RespuestaInicioSesion.
- RepositorioPanelUsuarioSupabase implementa CRUD de empleados y salarios via REST.

Flujo general de inicio de sesion

- 1) PantallaLogin recoge cedula y contrasena.
- 2) VistaLogin valida obligatorios.
- 3) Repositorio ejecuta login con PHP o Supabase.
- 4) Respuesta actualiza sesion y rol.

Raiz del paquete

- MainActivity.kt: pantalla principal que inicia la app.

componentes/

- ComponentesLogin.kt: piezas reutilizables del login.

config/

- BackendConfig.kt: elige si se usa PHP o Supabase.
- Contenedor.kt: registra y entrega dependencias comunes.
- ProveedorAuth.kt: crea el repositorio de autenticacion segun el backend.
- ProveedorDatos.kt: crea repositorios de panel y ubicaciones.

debug/

- DebugActivity.kt: pruebas internas.

- LoginActivity.kt: pruebas del login.

modelos/

- EstadoLogin.kt: estado y eventos del login.
- ModelosAuth.kt: estructuras de login (solicitud y respuesta).
- ModelosPanel.kt: datos de empleados y salarios.

pantallas/

- PantallaLogin.kt: pantalla de inicio de sesion.
- PantallaPanel.kt: panel principal (empleados, salarios, perfil).
- PantallaResumen.kt: pantalla de apoyo para pruebas.

red/

- ApiAuth.kt: endpoints de login para PHP.
- ApiUbicacion.kt: endpoints de ubicaciones.
- ClienteRed.kt: cliente Retrofit.
- ConfigRed.kt: URL y ajustes de red.

red/dto/

- AuthDto.kt: datos que viajan en login.
- PanelDto.kt: datos de panel/empleados/salarios.
- UbicacionDto.kt: datos de provincias, distritos y corregimientos.

repositorios/

- RepoAuth.kt: interfaz de autenticacion.
- RepoAuthMock.kt: autenticacion simulada.
- RepoAuthPhp.kt: autenticacion por PHP.
- RepoPanel.kt: interfaz del panel.
- RepoPanelPhp.kt: panel por PHP.
- RepositorioAutenticacionSupabase.kt: autenticacion por Supabase.
- RepositorioPanelUsuarioSupabase.kt: panel por Supabase.
- RepositorioUbicaciones.kt: interfaz de ubicaciones.
- RepositorioUbicacionesPhp.kt: ubicaciones por PHP.
- RepositorioUbicacionesSupabase.kt: ubicaciones por Supabase.

salario/

- CalculadoraSalario.kt: calcula salario y deducciones.

supabase/

- SupabaseApi.kt: endpoints REST de Supabase.
- SupabaseAuth.kt: utilidades de auth Supabase.
- SupabaseClient.kt: cliente HTTP con headers.
- SupabaseConfig.kt: URL y API key.
- SupabaseDtos.kt: datos para Supabase.
- SupabaseHashContrasena.kt: hash y verificacion de contrasena.
- SupabaseManager.kt: orquesta la integracion Supabase.

tema/

- Color.kt: colores del tema.
- Theme.kt: tema Material.
- Type.kt: tipografias.

validacion/

- ValidadorFormularioEmpleado.kt: reglas de empleados.
- ValidadorLogin.kt: reglas de login.

vistas/

- VistaLogin.kt: ViewModel del login.
- VistaPanel.kt: ViewModel del panel.